

Q1. Develop a program to create histograms for all numerical features and analyze the distribution of each feature. Generate box plots for all numerical features and identify any outliers. Use California Housing dataset.

```
import pandas as pd

import numpy as np

import seaborn as sns

import matplotlib.pyplot as plt

from sklearn.datasets import fetch_california_housing

def load_data():

    data = fetch_california_housing(as_frame=True)

    return data.frame

def plot_histograms(df, features):

    plt.figure(figsize=(15, 10))

    for i in range(len(features)):

        plt.subplot(3, 3, i + 1)

        sns.histplot(df[features[i]], kde=True, bins=30)

        plt.title("Distribution of " + features[i])

    plt.tight_layout()

    plt.show()

def plot_boxplots(df, features):

    plt.figure(figsize=(15, 10))

    for i in range(len(features)):

        plt.subplot(3, 3, i + 1)

        sns.boxplot(x=df[features[i]])
```

```

plt.title("Box Plot of " + features[i])
plt.tight_layout()
plt.show()

def detect_outliers(df, features):
    print("Outliers Detection:\n")
    for feature in features:
        Q1 = df[feature].quantile(0.25)
        Q3 = df[feature].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        outliers = df[(df[feature] < lower_bound) | (df[feature] > upper_bound)]
        print(feature + ":", len(outliers), "outliers")

if __name__ == "__main__":
    housing_df = load_data()
    numerical_features = housing_df.select_dtypes(include=[np.number]).columns
    plot_histograms(housing_df, numerical_features)
    plot_boxplots(housing_df, numerical_features)
    detect_outliers(housing_df, numerical_features)
    print("\nDataset Summary:\n")
    print(housing_df.describe())

```

Q2. Develop a program to Compute the correlation matrix to understand the relationships between pairs of features. Visualize the correlation matrix using a heatmap to know which variables have strong positive/negative correlations. Create a pair plot to visualize pairwise relationships between features. Use California Housing dataset.

```
import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

from sklearn.datasets import fetch_california_housing

# Step 1: Load the California Housing Dataset

california_data = fetch_california_housing(as_frame=True)

data = california_data.frame

# Step 2: Compute the correlation matrix

correlation_matrix = data.corr()

# Step 3: Visualize the correlation matrix using a heatmap

plt.figure(figsize=(10, 8))

sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt='.2f',
            linewidths=0.5)

plt.title('Correlation Matrix of California Housing Features')

plt.show()

# Step 4: Create a pair plot to visualize pairwise relationships

sns.pairplot(data, diag_kind='kde', plot_kws={'alpha': 0.5})

plt.suptitle('Pair Plot of California Housing Features', y=1.02)

plt.show()
```

Q3. Develop a program to implement Principal Component Analysis (PCA) for reducing the dimensionality of the Iris dataset from 4 features to 2.

```
import numpy as np

import pandas as pd

from sklearn.datasets import load_iris

from sklearn.decomposition import PCA

import matplotlib.pyplot as plt


# Load the Iris dataset

iris = load_iris()

data = iris.data

labels = iris.target

label_names = iris.target_names


# Convert to a DataFrame for better visualization

iris_df = pd.DataFrame(data, columns=iris.feature_names)


# Perform PCA to reduce dimensionality to 2

pca = PCA(n_components=2)

data_reduced = pca.fit_transform(data)


# Create a DataFrame for the reduced data

reduced_df = pd.DataFrame(data_reduced, columns=['Principal Component 1',
'Principal Component 2'])

reduced_df['Label'] = labels


# Plot the reduced data

plt.figure(figsize=(8, 6))

colors = ['r', 'g', 'b']
```

```
for i, label in enumerate(np.unique(labels)):

    plt.scatter(

        reduced_df[reduced_df['Label'] == label]['Principal Component 1'],
        reduced_df[reduced_df['Label'] == label]['Principal Component 2'],
        label=label_names[label],
        color=colors[i]
    )

plt.title('PCA on Iris Dataset')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend()
plt.grid()
plt.show()
```

Q4. For a given set of training data examples stored in a .CSV file, implement and demonstrate the Find-S algorithm to output a description of the set of all hypotheses consistent with the training examples.

- **Open new textfile and type this code and save it as csv file .**

Step1:

Outlook, Temperature, Humidity, Windy, PlayTennis

Sunny, Hot, High, False, No

Sunny, Hot, High, True, No

Overcast, Hot, High, False, Yes

Rain, Mild, High, False, Yes

Rain, Cool, Normal, False, Yes

Rain, Cool, Normal, True, No

Overcast, Cool, Normal, True, Yes

Sunny, Mild, High, False, No

Sunny, Cool, Normal, False, Yes

Rain, Mild, Normal, False, Yes

Sunny, Mild, Normal, True, Yes

Overcast, Mild, High, True, Yes

Overcast, Hot, Normal, False, Yes

Rain, Mild, High, True, No

- Then new notebook and paste this code and run by using shift +Enter

```
import pandas as pd
```

```
def find_s_algorithm(file_path):
```

```
    data = pd.read_csv(file_path)
```

```
    print("Training data:")
```

```
    print(data)
```

```
    attributes = data.columns[:-1]
```

```
    class_label = data.columns[-1]
```

```
    hypothesis = None
```

```
    for index, row in data.iterrows():
```

```
        if row[class_label] == "Yes":
```

```
            if hypothesis is None:
```

```
                hypothesis = list(row[attributes])
```

```
            else:
```

```
                for i in range(len(attributes)):
```

```
                    if hypothesis[i] != row.iloc[i]:
```

```
                        hypothesis[i] = "?"
```

```
return hypothesis
```

```
file_path = "training_data.csv"
```

```
hypothesis = find_s_algorithm(file_path)
```

```
print("\nThe final hypothesis is:", hypothesis)
```


Q5. Develop a program to implement k-Nearest Neighbour algorithm to classify the randomly generated 100 values of x in the range of [0,1]. Perform the following based on dataset generated.

- a) Label the first 50 points {x1,.....,x50} as follows: if $(x_i \leq 0.5)$, then $x_i \in \text{Class1}$, else $x_i \in \text{Class2}$**
- b) Classify the remaining points, x51,.....,x100 using KNN. Perform this for k=1,2,3,4,5,20,30**

```
import numpy as np

import matplotlib.pyplot as plt

from collections import Counter

# Step 1: Generate data

np.random.seed(42)

data = np.random.rand(100)

# Step 2: Split data

train_data = data[:50]

test_data = data[50:]

# Step 3: Label training data

train_labels = ["Class1" if x <= 0.5 else "Class2" for x in train_data]

# Step 4: KNN function

def knn(test_point, k):

    distances = [(abs(test_point - train_data[i]), train_labels[i]) for i in range(len(train_data))]

    distances.sort()

    neighbors = [label for _, label in distances[:k]]

    return Counter(neighbors).most_common(1)[0][0]

# Step 5: k values

k_values = [1, 2, 3, 4, 5, 20, 30]
```

```
# Step 6: Classification + Plot
```

```
for k in k_values:
```

```
    predictions = [knn(point, k) for point in test_data]
```

```
    print(f"\nResults for k = {k}")
```

```
    for i, pred in enumerate(predictions, start=51):
```

```
        print(f"x{i} = {test_data[i-51]:.4f} -> {pred}")
```

```
    plt.figure()
```

```
    # Training points
```

```
    colors = ["blue" if label == "Class1" else "red" for label in train_labels]
```

```
    plt.scatter(train_data, [0]*50, c=colors, label="Training")
```

```
    # Test points
```

```
    test_colors = ["blue" if p == "Class1" else "red" for p in predictions]
```

```
    plt.scatter(test_data, [1]*50, c=test_colors, marker='x', label="Test")
```

```
    plt.title(f"KNN (k = {k})")
```

```
    plt.xlabel("Data values")
```

```
    plt.yticks([0,1], ["Train", "Test"])
```

```
    plt.legend()
```

```
    plt.grid()
```

```
plt.show()
```

Q6. Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select appropriate data set for your experiment and draw graphs.

```
import numpy as np

import matplotlib.pyplot as plt

def gaussian_kernel(x, xi, tau):

    return np.exp(-np.sum((x - xi) ** 2) / (2 * tau ** 2))

def locally_weighted_regression(x, X, y, tau):

    m = X.shape[0]

    weights = np.array([gaussian_kernel(x, X[i], tau) for i in range(m)])

    W = np.diag(weights)

    X_transpose_W = X.T @ W

    theta = np.linalg.inv(X_transpose_W @ X) @ X_transpose_W @ y

    return x @ theta

np.random.seed(42)

X = np.linspace(0, 2 * np.pi, 100)

y = np.sin(X) + 0.1 * np.random.randn(100)

X_bias = np.c_[np.ones(X.shape), X]

x_test = np.linspace(0, 2 * np.pi, 200)

x_test_bias = np.c_[np.ones(x_test.shape), x_test]

tau = 0.5

y_pred = np.array([locally_weighted_regression(xi, X_bias, y, tau) for xi in x_test_bias])

plt.figure(figsize=(10, 6))

plt.scatter(X, y, color='red', label='Training Data', alpha=0.7)

plt.plot(x_test, y_pred, color='blue', label=f'LWR Fit (tau={tau})', linewidth=2)

plt.xlabel('X', fontsize=12)

plt.ylabel('y', fontsize=12)

plt.title('Locally Weighted Regression', fontsize=14)

plt.legend(fontsize=10)

plt.grid(alpha=0.3)
```

Q7. Develop a program to demonstrate the working of Linear Regression and Polynomial Regression. Use Boston Housing Dataset for Linear Regression and Auto MPG Dataset (for vehicle fuel efficiency prediction) for Polynomial Regression.

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from sklearn.datasets import fetch_openml

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn.preprocessing import PolynomialFeatures

from sklearn.metrics import mean_squared_error, r2_score

print("Linear Regression - Boston Housing")

data = fetch_openml(name="boston", version=1, as_frame=True)

df = data.frame

X = df[["RM"]]

y = df["MEDV"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("MSE:", mean_squared_error(y_test, y_pred))

print("R2:", r2_score(y_test, y_pred))

plt.scatter(X_test, y_test, color="blue")

plt.plot(X_test, y_pred, color="red")

plt.title("Linear Regression - Boston Housing")

plt.xlabel("Rooms")

plt.ylabel("Price")

plt.show()
```

```

print("\nPolynomial Regression - Auto MPG")

url = "https://archive.ics.uci.edu/ml/machine-learning-databases/auto-mpg/auto-mpg.data"

columns = ["mpg","cylinders","displacement","horsepower","weight",
           "acceleration","model_year","origin"]

df = pd.read_csv(url, sep='\s+', names=columns, na_values="?").dropna()

X = df[["horsepower"]]

y = df["mpg"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

poly = PolynomialFeatures(degree=2)

X_train_poly = poly.fit_transform(X_train)

X_test_poly = poly.transform(X_test)

model = LinearRegression()

model.fit(X_train_poly, y_train)

y_pred = model.predict(X_test_poly)

print("MSE:", mean_squared_error(y_test, y_pred))

print("R2:", r2_score(y_test, y_pred))

plt.scatter(X_test, y_test, color="blue")

X_range = pd.DataFrame(
    np.linspace(X["horsepower"].min(), X["horsepower"].max(), 100),
    columns=["horsepower"]
)

X_range_poly = poly.transform(X_range)

y_range = model.predict(X_range_poly)

plt.plot(X_range, y_range, color="red")

plt.title("Polynomial Regression - Auto MPG")

plt.xlabel("Horsepower")

plt.ylabel("MPG")

plt.show()

```

Q8 .Develop a program to demonstrate the working of the decision tree algorithm. Use Breast Cancer Data set for building the decision tree and apply this knowledge to classify a new sample.

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score

from sklearn import tree

data = load_breast_cancer()

X = data.data

y = data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

clf = DecisionTreeClassifier(random_state=42)

clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print(f"Model Accuracy: {accuracy * 100:.2f}%")

new_sample = np.array([X_test[0]])

prediction = clf.predict(new_sample)

prediction_class = "Benign" if prediction == 1 else "Malignant"

print(f"Predicted Class for the new sample: {prediction_class}")

plt.figure(figsize=(12,8))

tree.plot_tree(clf, filled=True, feature_names=data.feature_names,
class_names=data.target_names)

plt.title("Decision Tree - Breast Cancer Dataset")

plt.show()
```

Q9. Develop a program to implement the Naive Bayesian classifier considering Olivetti Face Data set for training. Compute the accuracy of the classifier, considering a few test data sets.

```
import numpy as np

from sklearn.datasets import fetch_olivetti_faces

from sklearn.model_selection import train_test_split, cross_val_score

from sklearn.naive_bayes import GaussianNB

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

import matplotlib.pyplot as plt

data = fetch_olivetti_faces(shuffle=True, random_state=42)

X = data.data

y = data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

gnb = GaussianNB()

gnb.fit(X_train, y_train)

y_pred = gnb.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print(f'Accuracy: {accuracy * 100:.2f}%')

print("\nClassification Report:")

print(classification_report(y_test, y_pred, zero_division=1))

print("\nConfusion Matrix:")

print(confusion_matrix(y_test, y_pred))

cross_val_accuracy = cross_val_score(gnb, X, y, cv=5, scoring='accuracy')

print(f'\nCross-validation accuracy: {cross_val_accuracy.mean() * 100:.2f}%')

fig, axes = plt.subplots(3, 5, figsize=(12, 8))

for ax, image, label, prediction in zip(axes.ravel(), X_test, y_test, y_pred):

    ax.imshow(image.reshape(64, 64), cmap=plt.cm.gray)

    ax.set_title(f"True: {label}, Pred: {prediction}")

    ax.axis('off')
```

Q10. . Develop a program to implement k-means clustering using Wisconsin Breast Cancer data set and visualize the clustering result.

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.datasets import load_breast_cancer

from sklearn.cluster import KMeans

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA

from sklearn.metrics import confusion_matrix, classification_report

data = load_breast_cancer()

X = data.data

y = data.target

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

kmeans = KMeans(n_clusters=2, random_state=42)

y_kmeans = kmeans.fit_predict(X_scaled)

print("Confusion Matrix:")

print(confusion_matrix(y, y_kmeans))

print("\nClassification Report:")

print(classification_report(y, y_kmeans))

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)

df = pd.DataFrame(X_pca, columns=['PC1', 'PC2'])

df['Cluster'] = y_kmeans

df['True Label'] = y
```



```

plt.figure(figsize=(8, 6))

sns.scatterplot(data=df, x='PC1', y='PC2', hue='Cluster', palette='Set1', s=100,
edgecolor='black', alpha=0.7)

plt.title('K-Means Clustering of Breast Cancer Dataset')

plt.xlabel('Principal Component 1')

plt.ylabel('Principal Component 2')

plt.legend(title="Cluster")

plt.show()

plt.figure(figsize=(8, 6))

sns.scatterplot(data=df, x='PC1', y='PC2', hue='True Label', palette='coolwarm', s=100,
edgecolor='black', alpha=0.7)

plt.title('True Labels of Breast Cancer Dataset')

plt.xlabel('Principal Component 1')

plt.ylabel('Principal Component 2')

plt.legend(title="True Label")

plt.show()

plt.figure(figsize=(8, 6))

sns.scatterplot(data=df, x='PC1', y='PC2', hue='Cluster', palette='Set1', s=100,
edgecolor='black', alpha=0.7)

centers = pca.transform(kmeans.cluster_centers_)

plt.scatter(centers[:, 0], centers[:, 1], s=200, c='red', marker='X', label='Centroids')

plt.title('K-Means Clustering with Centroids')

plt.xlabel('Principal Component 1')

plt.ylabel('Principal Component 2')

plt.legend(title="Cluster")

plt.show()

```

